

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב

67130 – מבוא למדעי המחשב בדגש על תכנות פרוצדוראלי

תאריך: 30.10.2008

משך הבחינה: שלוש שעות

מועד ג' - סמסטר א' - תשס"ח – 2007-2008

המורה: פרופ' ג'ף רוזנשיין

הנחיות:

- יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (33 נק').
- יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת נפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.
- יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
- הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
- יש למחוק טיוטות באופן ברור.
- יש להשאיר שוליים.
- אסור להשתמש בעט אדום.
- הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
- תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
- פתרון מיטבי לשאלה יחשב פתרון **פשוט ויעיל ככל האפשר**. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
- אין להקצות** או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה. עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להקצות מערך בן שלשה תאים פעם אחת).
- בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. כמו כן, ניתן להניח כי המערכים והרשימות המקושרות הנתונות מכילים ערכים תקינים ואינם NULL, או מערכים בגודל 0 אלא אם מבהר בשאלה אחרת.

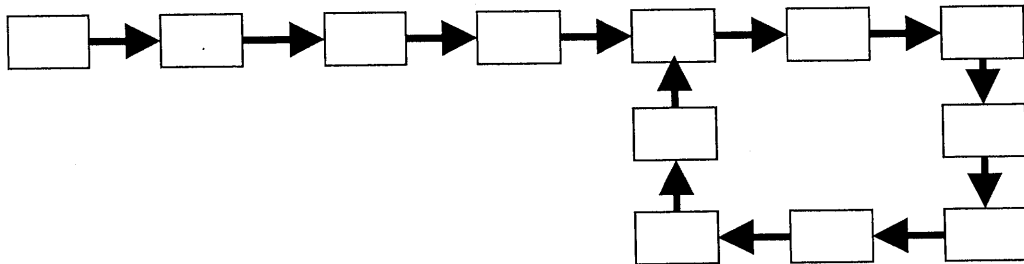
שאלה 1 (33 נקודות)

מציאת מעגל ברשימה מקושרת (חד כיוונית):

נתונה המחלקה Node, אותה כבר מימשנו עבורכם ובה השיטה:

`Node getNext()`

דוגמה לרשימה המכילה מעגל:



א. (17 נקודות)

כיתבו שיטה `public static boolean findCycle(Node head)` המקבלת מצביע לראש של רשימה ומחזירה ערך בוליאני - `true` במקרה שהרשימה מכילה מעגל ו-`false` אם הרשימה אינה מכילה מעגל. אין לשנות את הרשימה המקורית. ניתן להשתמש רק בכמות קבועה של זיכרון (שאינה תלויה במספר ה-`nodes` ברשימה). על השיטה לעבוד בסיבוכיות זמן $O(n^2)$ שאינה גם $O(n)$ (כאשר n מסמן את מספר ה-`nodes` ברשימה). הראו שעמדתם בדרישה זו.

ב. (16 נקודות)

כעת כיתבו שיטה נוספת `public static boolean findCycle(Node head)` המקבלת מצביע לראש של רשימה ומחזירה ערך בוליאני - `true` במקרה שהרשימה מכילה מעגל ו-`false` אם הרשימה אינה מכילה מעגל. שוב, אין לשנות את הרשימה המקורית, וניתן להשתמש רק בכמות קבועה של זיכרון. אולם הפעם, על השיטה לעבוד בסיבוכיות זמן $O(n)$. הראו שעמדתם בדרישה זו.

רמז: אכילס והצב עורכים תחרות ריצה מסביב לאיצטדיון האולימפי של אתונה. אכילס קל הרגליים רץ פי שניים מהר מהצב הסטלן. כמה פעמים יעקוף אכילס את הצב במהלך הקפה אחת? כמה פעמים יעקוף אותו בתחרות של n הקפות? האם מספר העקיפות תלוי בהיקף האיצטדיון?

67. 101 - 67. 130
2 / 40 P1

שאלה 2 (33 נקודות)

נתון מערך חד מימדי שברצונכם למיין בסדר יורד (מהערכים הגבוהים אל הנמוכים). לדוגמה הנה המערך הנתון בתחילה:

18	35	22	97	84	55	61	10	47
----	----	----	----	----	----	----	----	----

והמערך שצריך להתקבל בסוף המיין הוא:

97	84	61	55	47	35	22	18	10
----	----	----	----	----	----	----	----	----

שיפור אפשרי לאלגוריתם של Bubble Sort הוא האלגוריתם שנקרא Cocktail Sort. באלגוריתם זה מבצעים תחילה מעבר אחד בכיוון ימין על המערך (בו מבצעים לצד ימין את האיבר הקטן ביותר) ולאחריו מבצעים מעבר על המערך בכיוון שמאל (בו מבצעים לצד שמאל את האיבר הגדול). שני מעברים אלו מהווים שלב אחד באלגוריתם. בשלב הבא המעברים יביאו את האיבר השני הכי קטן והאיבר השני בגודלו אל מקומם הנכון במערך וכן הלאה. אחרי i מעברים, i האיברים הקטנים ביותר ו- i האיברים הגדולים ביותר ימוקמו במקום הנכון במערך, ואין צורך לבדוק אותם בעת ביצוע מעברים נוספים.

א. (14 נקודות)

ממשו את השיטה `public static void cocktailSort (int [ ] a)`

הממיינת את המערך הנתון בעזרת אלגוריתם Cocktail Sort שתיארנו למעלה.

שימו לב שאתם מממשים גרסה משופרת של Bubble Sort ולא גרסה של Selection Sort או של Insertion

Sort. המימוש שלכם צריך להפסיק לרוץ בלולאה ברגע שהמערך כבר ממוין ולא להמשיך למיין במידה והמערך

ממוין לפני שביצעתם את כל המעברים.

ב. (3 נקודות)

מהי הבעייה אותה פותר אלגוריתם Cocktail Sort? כלומר, באיזה מצב יעבוד Cocktail Sort טוב

יותר מאשר Bubble Sort?

ג. (2 נקודות)

מהי סיבוכיות זמן הריצה (במונחי big-O) של מתודת `cocktailSort()` הסבר!

ד. (14 נקודות)

שיפור נוסף אפשרי ל-Bubble Sort הוא אלגוריתם Comb Sort המתנהג בדומה ל-Bubble Sort מלבד העובדה

שהוא מבצע השוואות והחלפות בין איברים שהם במרחק n אחד מהשני, כאשר n אינו בהכרח 1 (כלומר האיברים

אינם בהכרח סמוכים). אחרי ביצוע מעבר אחד, ערכו של n קטן פי קבוע המכונה "פקטור הכיווץ". המעבר הבא

מבוצע עם הערך החדש של n , ולאחריו n שוב קטן באותו אופן. כך נמשכים המעברים עד אשר n מגיע לערך 1.

בשלב זה ממשיכים לבצע מיוני Bubble Sort רגילים ועוצרים ברגע שהמערך כבר ממוין.

כתבו שיטה `public static void combSort (int [ ] a)`

המממשת את אלגוריתם Comb Sort. התחילו עם n שערכו שווה לאורך המערך חלקי 1.3 (עגלו כלפי מטה)

והשתמשו בפקטור כיווץ של 1.3 לאחר כל מעבר. לאחר ש- n מגיע ל-1 יש להפסיק להקטין אותו ולבדוק בכל מעבר

אם המערך כבר ממוין (ואז כמובן להפסיק למיין).

67.101-67.130
 'a / 4'OPA

שאלה 3 (33 נקודות)

איילת הגנת רוצה להושיב את הילדים בגן על הספסל, אך כל ילד מוכן לשבת רק על יד חבריו. עזרו לה:
 בהינתן הטיפוס *Child*, אשר לו שיטה -

public boolean isFriend(Child other)

המחזירה *true* אם ורק אם הילד המיוצג על ידי *this* אוהב את הילד אשר מיוצג על ידי *other*.

עליך לכתוב שיטה **רקורסיבית** - **public boolean findSeating(Child []children)**

המנסה למצוא סידור חוקי לילדים במערך *children* (המייצג בעצם את הספסל), כך שהילד המיוצג על ידי *children[i]* יזוהה על ידי השיטה *isFriend* כחבר של הילד המיוצג על ידי *children[i+1]* (עבור כל $0 \leq i \leq \text{children.length}-2$).

אם קיים סידור אפשרי של הילדים על השיטה להחזיר *true*, ולסדר את האברים במערך על פי הסידור הנבחר (דמו את המערך לספסל). אם לא קיים סידור, השיטה תחזיר *false*, וסדר האברים במערך לא ישונה.
 לדוגמה, בהינתן טבלת החברויות מימין, הטבלה משמאל מייצגת שני סידורים חוקיים של הילדים, וסידור אחד לא חוקי, משום שאין קשר חברות בין **דני דוד**.

סידור				חוקי
				לא
				חוקי
דני	משה	יוסי	דוד	חוקי
דני	משה	דוד	יוסי	חוקי
דוד	דני	משה	יוסי	לא
				חוקי

עם	משה	דוד	יוסי	דני
משה	X	X	X	X
דוד		X	X	X
יוסי			X	X
דני				X

על כן, עליכם לפעול לפי ההנחות הבאות:

- קשר החברות הוא סימטרי: אם משה חבר של דוד, הינכם יכולים להניח כי דוד חבר של משה.
- קשר החברות אינו טרנזיטיבי: אם דוד חבר של משה, ומשה חבר של דני, לא בטוח שדוד חבר של דני...
- אם קיימים מספר סידורים חוקיים, יש לבחור אחד מהם.
- לא בהכרח קיים סידור חוקי.

א. (3 נקודות)

תארו בקצרה (חמש שורות מלל לכל היותר) את הפתרון שלכם (אשר יובא במלואו בסעיף 2).

ב. (25 נקודות)

ממשו את השיטה *findSeating* (ניתן להגדיר שיטות עזר).

ג. (5 נקודות)

מהו זמן הריצה של הפתרון שמימשתם? הסבירו!

67. 101 - 67. 130
 '2 / 40 R1

שאלה 4 (33 נקודות)

מכונה אוטומטית מוכרת שני סוגי פריטים: קוביות וכדורים. המחיר של פריט תלוי בנפחו: קוביות נמכרות עבור 1 דולר לס"מ³, וכדורים נמכרים עבור $3/\pi$ דולר לס"מ³. חלק מהפריטים "משומשים" ונמכרים בהנחה של 10%. המכונה מאחסנת פריטים במדיניות Last In First Out, כלומר הפריטים נמכרים בסדר הפוך לסדר אחסונם במכונה.

א. (15 נקודות)

כתבו מחלקה אבסטרקטית בשם `Item` המייצגת פריט למכירה. כתבו שתי מחלקות `CubeItem`, `SphereItem` היורשות מהמחלקה `Item` ומייצגות קוביות וכדורים בהתאמה. (עליך להחליט מה יהיה הממשק של כל אחת מהמחלקות ואיזו פונקציונאליות תמומש בכל מחלקה).

ב. (10 נקודות)

כתבו מחלקה בשם `ItemStack` המייצגת מחסנית של פריטים (בגודל חסום). (רמז: השתמשו במערך). המחלקה צריכה לממש את השיטות הבאות:

<code>public ItemStack(int size)</code>	בונה מחסנית חדשה המסוגלת לאחסן עד <code>size</code> פריטים.
<code>public void push(Item item)</code>	דוחפת את הפריט הנתון לראש המחסנית. (אם המחסנית כבר מלאה, לא נעשה דבר).
<code>public Item pop()</code>	מוציאה את הפריט שבראש המחסנית. (אם המחסנית ריקה, מוחזר <code>null</code>).

ג. (8 נקודות)

כתבו מחלקה בשם `VendingMachine` המייצגת את המכונה. המחלקה צריכה לממש את השיטות הבאות:

<code>public VendingMachine(int size)</code>	בונה מכונה חדשה המסוגלת לאחסן עד <code>size</code> פריטים.
<code>public void stockCube(double edgeLength, boolean used)</code>	מאחסנת במכונה קובייה עם צלע באורך <code>edgeLength</code> ס"מ. הפרמטר <code>used</code> מציינ האם הפריט משומש. (אם המכונה כבר מלאה, לא נעשה דבר).
<code>public void stockSphere(double radius, boolean used)</code>	מאחסנת במכונה כדור ברדיוס <code>radius</code> ס"מ. הפרמטר <code>used</code> מציינ האם הפריט משומש. (אם המכונה כבר מלאה, לא נעשה דבר).
<code>public void buy()</code>	מוציאה מהמכונה את הפריט הבא למכירה, ומדפיסה את ההודעה: "Thank you for buying a \$ <i>p</i> <i>i</i> " כאשר <i>p</i> הוא מחיר הפריט, <i>i</i> הוא שם הפריט (cube או sphere). (אם המכונה ריקה, לא נעשה דבר).

67. 101 - 67. 130
'd / 4'0P1

להלן דוגמא לקטע תוכנית המשתמש במחלקה VendingMachine:

```
VendingMachine machine = new VendingMachine(5);  
machine.stockCube(2.0, true);  
machine.stockSphere(1.0, false);  
machine.buy();  
machine.buy();
```

הפלט של קטע זה הוא:

```
Thank you for buying a $4.0 sphere  
Thank you for buying a $7.2 cube
```

תזכורת: נפחה של קובייה עם צלע באורך x הוא x^3 . נפחו של כדור ברדיוס r הוא $\frac{4}{3} \pi r^3$. ניתן להשתמש בקבוע Math.PI על מנת לקבל את ערכו של π .

ב ה צ ל ה !